

Data Mastery 120: Roadmap Integral de Minería y Análisis de Datos

Día 2 -Fundamentos SQL y Normalización



Objetivo del Día

- Dominar los fundamentos de **SQL** (**SELECT**, **WHERE**, **ORDER BY**).
- Comprender y aplicar las **formas normales** (1FN, 2FN, 3FN) y la **integridad referencial** (PK, FK).
- Diseñar un **esquema normalizado** a partir del **dataset IBM HR**.
- Ejecutar **queries** de filtrado y cargar datos masivos con **COPY**.



Diccionario Técnico Operativo

Elemento → Descripción → Sintaxis → Caso práctico → Pitfall



SELECT:

- Extrae datos de una **tabla**.
- **Sintaxis**:

```
SELECT
    columna1,
    columna2
FROM tabla;
```

- **Pitfall**: olvidar ; al final.
- **Caso 1: Vista rápida de empleados senior**

```
/*****
*****
Día 2 - 120    Diccionario Técnico Operativo    Caso 01: SELECT
*****
*****/
```

```
/*****
*****
Título: Reporte de Empleados Senior
```

Objetivo: Identificar a los empleados con una edad superior a 50 años.

Descripción: Este script selecciona el número de empleado, su edad y su puesto

de trabajo de la tabla maestra de empleados, filtrando únicamente aquellos cuya

edad sea mayor a 50 años para análisis de talento senior.

Resultado CSV: day02_senior_empl_over_50.csv

```
*****
*****/
```

-- Selecciona los datos necesarios de la tabla maestra de empleados.

```

SELECT
    e.employee_number, -- Identificador único del empleado
    e.age,              -- Edad actual del empleado
    e.job_role-- Puesto o rol que desempeña
FROM
    employee_master_data AS e
WHERE
    e.age > 50-- Filtra solo a empleados con edad mayor a 50 años
ORDER BY
    e.age DESC;-- Ordena resultados de mayor a menor edad

```

- **Objetivo:** Identificar empleados senior.
- **Valor:** Detectar perfiles con experiencia para programas de mentoría.
- **Insights:**
 - **Concentración de Liderazgo y Mentoría:** Los roles críticos (**Manager, Research Director, Healthcare Representative**) se concentran en personal de 55-60 años, garantizando **estabilidad** y custodia de la **cultura organizacional**. Se recomienda aprovechar a este talento experimentado como **mentores** clave.
 - **Alerta de Relevo Generacional:** Empleados clave (#549, #1697, #573) cercanos a los 60 años entran en fase de **jubilación**. Esto genera un **riesgo** alto de pérdida de **capital intelectual** y **conocimiento especializado**, exigiendo **planes de sucesión** urgentes.
 - **Talento Senior Operativo y Retención:** Contamos con una fuerza laboral **+50 años** diversa que combina **roles directivos** con **especialistas técnicos** (técnicos, científicos) que garantizan alta **fiabilidad operativa** y **conocimiento histórico**. Tener un volumen alto de personal senior es un indicador de **estabilidad**, fortalece la **marca empleadora** y demuestra **retención de largo plazo** al valorar la **lealtad** y **experiencia**, lo que reduce la **rotación** de personal.
- **Resumen Estratégico:**
Para gestionar el retiro de **talento senior**, se requiere un **Programa de Sucesión Crítica** (empleados 58-60 años). El objetivo es documentar el conocimiento y formar **sucesores** internos, garantizando la **continuidad operativa** sin sobresaltos.

employee_number	age	job_role
549	60	Manager
1697	60	Healthcare Representative
573	60	Sales Executive
732	60	Sales Executive
1233	60	Sales Executive
321	59	Human Resources
91	59	Sales Executive
10	59	Laboratory Technician
140	59	Manager
1283	59	Manufacturing Director
1254	59	Sales Executive
1048	59	Manager
81	59	Sales Executive

○ Caso 2: Selección de métricas clave

```
/*
*****
Día 2 - 120    Diccionario Técnico Operativo    Caso 02: SELECT
*****
*****/
```

```
/*
*****
```

Título: Extracción de Datos Maestros de Empleados para Evaluación

Objetivo: Obtener información clave de los empleados para analizar su desempeño en relación con sus ingresos mensuales.

Descripción: El script selecciona el número de empleado, su ingreso mensual y su calificación de desempeño actual desde la tabla maestra.

Resultado CSV: day02_empl_performance_vs_income.csv

```
*****
*****/
```

-- Selecciona las columnas específicas: id del empleado, sueldo y nota de desempeño

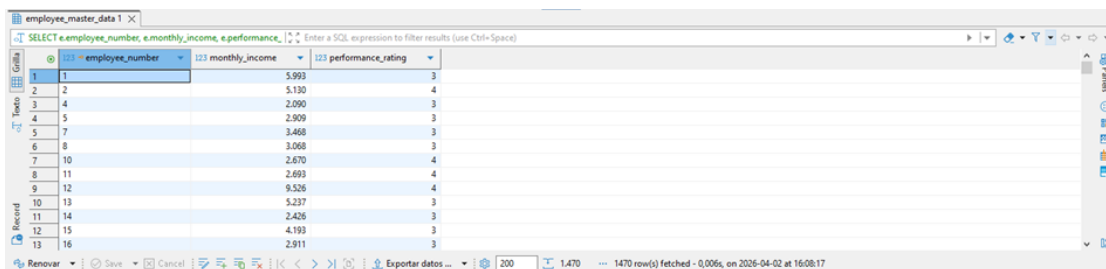
SELECT

e.employee_number, -- Identificador único del empleado
e.monthly_income, -- Salario base mensual
e.performance_rating -- Calificación del desempeño actual

FROM

employee_master_data AS e; -- Tabla origen con la información maestra de empleados

- **Objetivo:** Extraer métricas de desempeño y compensación.
- **Valor:** Relacionar ingresos con evaluaciones de desempeño.



The screenshot shows a database query result in a table with 4 columns: employee_number, monthly_income, and performance_rating. The table contains 16 rows of data. The status bar at the bottom indicates 1470 rows fetched.

	employee_number	monthly_income	performance_rating
1	1	5.993	3
2	2	5.130	4
3	4	2.090	3
4	5	2.909	3
5	7	3.468	3
6	8	3.068	3
7	10	2.670	4
8	11	2.693	4
9	12	9.526	4
10	13	5.237	3
11	14	2.426	3
12	15	4.193	3
13	16	2.911	3

○ Caso 3: Selección de columnas específicas para Power BI

```
/*  
*****  
Día 2 - 120    Diccionario Técnico Operativo    Caso 03: SELECT  
*****  
*****/
```

```
/*  
*****
```

Título: Análisis de Rotación por Departamento y Puesto

Objetivo: Identificar los niveles de rotación de personal (attrition) desglosados por departamento y rol laboral.

Descripción: Este script extrae datos de la tabla maestra de empleados para mostrar qué departamentos y puestos específicos están experimentando salidas de personal, permitiendo analizar tendencias de retención.

Resultado CSV: day02_empl_attrition_by_dept.csv

```
*****  
*****/
```

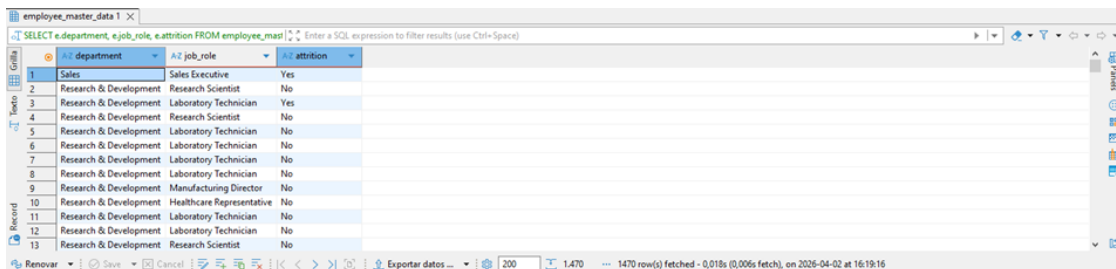
-- Selecciona las columnas clave: Departamento, Puesto y Estado de Rotación

SELECT

e.department, -- Nombre del área funcional
e.job_role, -- Puesto específico del empleado
e.attrition-- Indicador de si el empleado se fue (Sí/No)

FROM

employee_master_data AS e;-- Tabla origen con la información consolidada de empleados



	department	job_role	attrition
1	Sales	Sales Executive	Yes
2	Research & Development	Research Scientist	No
3	Research & Development	Laboratory Technician	Yes
4	Research & Development	Research Scientist	No
5	Research & Development	Laboratory Technician	No
6	Research & Development	Laboratory Technician	No
7	Research & Development	Laboratory Technician	No
8	Research & Development	Laboratory Technician	No
9	Research & Development	Manufacturing Director	No
10	Research & Development	Healthcare Representative	No
11	Research & Development	Laboratory Technician	No
12	Research & Development	Laboratory Technician	No
13	Research & Development	Research Scientist	No

- **Objetivo:** Preparar dataset reducido para visualización.
- **Valor:** Simplificar carga en Power BI.



WHERE:

- Filtra **registros** según **condición**.
- **Sintaxis:**

```
SELECT *
FROM tabla
WHERE condición;
```

- **Pitfall:** usar **=** en vez de **LIKE** para **texto**.
- **Caso 1: Empleados con riesgo de rotación**

```
/*****
****
Día 2 - 120    Diccionario Técnico Operativo    Caso 01: WHERE
****
*****/
```

```
/*****
****
Título: Análisis de Empleados con Rotación (Attrition)
```

Objetivo: Identificar qué empleados han abandonado la empresa para facilitar estudios sobre las causas de la rotación de personal.

Descripción: Este script selecciona el número de empleado, su departamento y el estado de rotación de la tabla 'employee_master_data', filtrando únicamente a aquellos que han abandonado la compañía ('Yes').

Resultado CSV: day02_attrition_cases_by_dept.csv

```
****
*****/
```

-- Selecciona las columnas clave para identificar al empleado y su área

SELECT

```
e.employee_number, -- Identificador único del empleado
e.department,      -- Área a la que pertenece
e.attrition-- Indica si el empleado se fue (Yes/No)
```

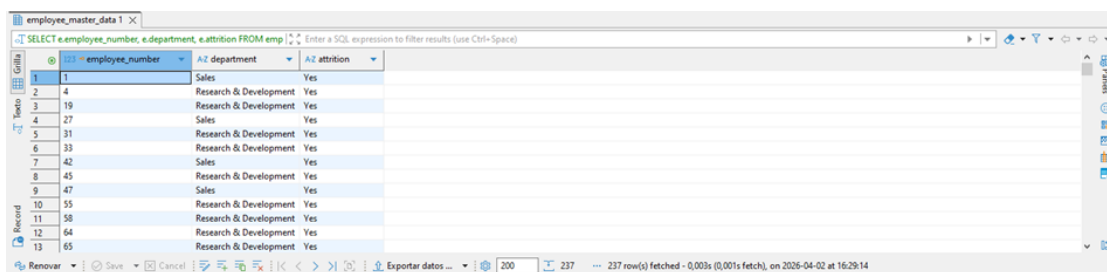
FROM

```
employee_master_data AS e-- Tabla principal de datos maestros
de empleados
```

WHERE

`e.attrition = 'Yes';`-- Filtra para mostrar solo a quienes abandonaron la empresa

- **Objetivo:** Identificar empleados que abandonaron la empresa.
- **Valor:** Analizar causas de rotación.
- **Insights:**
 - **Fuga de Talento: Research & Development (R&D)** es el mayor foco de **fuga de talento**, generando altos costos de aprendizaje y **riesgos técnicos**. La falta de retención pone en peligro la **propiedad intelectual** y los plazos de **lanzamiento**.
 - **Desgaste en la Fuerza de Ventas:** La **rotación** en **ventas** afecta directamente la **cartera de clientes** y la **rentabilidad**. La **baja de ejecutivos** evidencia **presión excesiva** o incentivos ineficaces que requieren atención inmediata.
 - **Estabilidad Relativa en Soporte (Human Resources):** Las funciones de **soporte** son más estables, pero el equipo de **RRHH** es pequeño: una **baja** crítica (como la #133) afecta la **gestión de plantilla**.
 - **Impacto en la Marca Empleadora:** Alto volumen de **bajas** alerta sobre la **retención de talento** y daña la **marca empleadora**. La **inestabilidad** percibida dificulta la **atracción** de personal, requiriendo analizar el **clima**, **carrera interna** o **competencia**.
- **Resumen Estratégico:** Iniciamos la **estrategia de retención** para frenar la pérdida de talento en **Research & Development (R&D)** y **Ventas**. No basta con registrar bajas; el siguiente paso es **analizar** las causas cruzando **satisfacción** y **antigüedad** para perfilar al **desertor**. Debemos pasar de la **actitud reactiva** a una **analítica predictiva** que **salve el talento** de la organización.



employee_number	department	attrition
1	Sales	Yes
2	Research & Development	Yes
3	Research & Development	Yes
4	Sales	Yes
5	Research & Development	Yes
6	Research & Development	Yes
7	Sales	Yes
8	Research & Development	Yes
9	Sales	Yes
10	Research & Development	Yes
11	Research & Development	Yes
12	Research & Development	Yes
13	Research & Development	Yes

○ Caso 2: Filtrar por ingresos y satisfacción

/*****

*****/

/*****

Título: Análisis de Empleados con Salario Alto y Baja Satisfacción

Objetivo: Detectar perfiles que ganan más de 5000 (unidad monetaria) pero tienen una puntuación de satisfacción laboral inferior a 2 (baja), posiblemente para evaluar riesgo de fuga o mejorar el clima laboral.

Descripción: Este script selecciona el número de empleado, su ingreso mensual y su nivel de satisfacción laboral de la tabla maestra de empleados.

Resultado CSV: day02_empl_retention_risk.csv

*****/

-- Seleccionamos los datos relevantes del empleado

SELECT

e.employee_number, -- Número identificador único del empleado
e.monthly_income, -- Ingreso mensual para identificar el nivel de salario
e.job_satisfaction-- Nivel de satisfacción (usado para detectar riesgo de fuga)

FROM

employee_master_data AS e-- Buscamos en la tabla maestra de empleados

WHERE

e.monthly_income > 5000-- FILTRO 1: Empleados con sueldo superior a 5000

AND e.job_satisfaction < 2-- FILTRO 2: Solo aquellos con baja satisfacción (1 o 0)

- **Objetivo:** Detectar empleados con alto salario pero baja satisfacción.
- **Valor:** Riesgo de fuga de talento clave.
- **Insights:**
 - **Ineficiencia del "Bono de Retención" Invisible:** Varios **empleados** con **sueldos altos** (\$18k-\$19k) muestran **insatisfacción** profunda. El **dinero no garantiza compromiso**. Pagamos **salarios de élite** por **productividad en riesgo**, resultando en un **bajo retorno de inversión (ROI)** de lealtad si se marchan.
 - **Zona de Peligro:** Personal **caro** y muy **atractivo para el mercado**. Perfiles técnicos/liderazgo con sueldos >\$5,000 en nivel 1 de

satisfacción están en **renuncia silenciosa**. Perderlos implica altos **costos de reemplazo** y **pérdida de conocimiento**.

- **Fuga de Conocimiento Especializado**: Al revisar los números, la **fuga de conocimiento** afecta a múltiples rangos. Existe **talento senior** de alto nivel desconectado de la visión empresarial. Esta desconexión en niveles altos provoca un **efecto dominó**, donde líderes insatisfechos dañan el **clima laboral** de todo su equipo.
- **Priorización de Intervención (Quick Wins)**: El reporte permite segmentar para **retener talento clave** primero. El negocio debe priorizar entrevistas con perfiles de alto salario (\$15k-\$19k). Es más barato ajustar condiciones y desafíos profesionales a un gerente actual que **reclutar** y capacitar uno nuevo.
- **Resumen de Impacto de Negocio**: Riesgo crítico de fuga de **capital intelectual** y **activos estratégicos** clave hacia la competencia.
- **Recomendación**: El salario no es el problema. Implementar acciones urgentes en **cultura**, **reconocimiento** y **balance vida-trabajo** para aumentar la satisfacción y retener el talento.

employee_number	monthly_income	job_satisfaction
20	9.900	1
38	18.947	1
52	5.376	1
68	5.454	1
70	9.884	1
74	9.069	1
81	7.637	1
86	9.724	1
101	13.245	1
104	10.673	1
114	7.484	1
153	11.631	1
170	6.567	1

○ Caso 3: Filtrar por campo educativo

```

/*****
*****
Día 2 - 120   Diccionario Técnico Operativo   Caso 03: WHERE
*****
*****/

```

```

/*****
*****

```

Título: Extracción de Empleados del Área de Ciencias de la Vida

Objetivo: Obtener una lista de empleados que cuentan con formación académica en el campo de 'Life Sciences'.

Descripción: Este script consulta la tabla maestra de empleados para filtrar

y seleccionar los números de identificación y el campo de estudio de aquellos cuyo perfil educativo sea 'Life Sciences'.

Resultado CSV: day02_life_sciences_talent_pool.csv

```
*****  
*****/
```

-- Selecciona las columnas identificador del empleado y campo educativo

SELECT

e.employee_number,
e.education_field

FROM

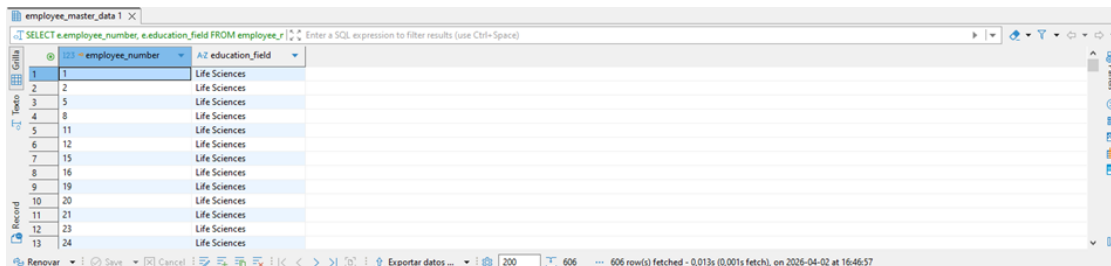
-- De la tabla principal de datos maestros de empleados

employee_master_data AS e

-- Filtra para incluir solamente a empleados con estudios en 'Life Sciences'

WHERE

e.education_field = 'Life Sciences';



employee_number	education_field
1	Life Sciences
2	Life Sciences
3	Life Sciences
4	Life Sciences
5	Life Sciences
6	Life Sciences
7	Life Sciences
8	Life Sciences
9	Life Sciences
10	Life Sciences
11	Life Sciences
12	Life Sciences
13	Life Sciences
14	Life Sciences
15	Life Sciences
16	Life Sciences
17	Life Sciences
18	Life Sciences
19	Life Sciences
20	Life Sciences
21	Life Sciences
22	Life Sciences
23	Life Sciences
24	Life Sciences

- **Objetivo:** Analizar perfiles académicos específicos.
- **Valor:** Evaluar correlación entre formación y desempeño.



ORDER BY:

- Ordena **resultados**.
- **Sintaxis:**

SELECT *
FROM tabla
ORDER BY columna **ASC|DESC;**

- **Pitfall:** no especificar **ASC/DESC** y asumir orden correcto.
- **Caso 1: Ranking de ingresos**

```
/*****  
*****
```

*****/

/*****

Título: Reporte de Ingresos Mensuales por Empleado

Objetivo: Identificar a los empleados con mayores ingresos mensuales.

Descripción: Este script consulta la base de datos maestra de empleados para obtener su número de identificación y su sueldo mensual, ordenando los resultados de mayor a menor ingreso para visualizar a los empleados mejor pagados primero.

Resultado CSV: day02_empl_monthly_income_ranking.csv

*****/

-- Selección de columnas: Número de empleado e ingresos mensuales

SELECT

e.employee_number,
e.monthly_income

-- Origen de datos: Tabla maestra de empleados

FROM

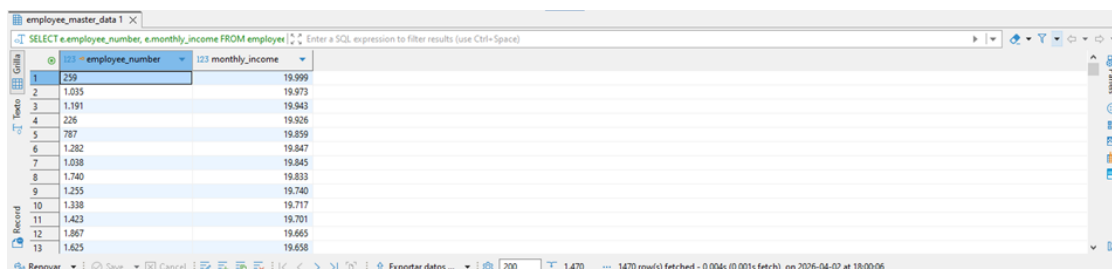
employee_master_data **AS** e

-- Ordenamiento: De mayor a menor (DESC) basado en el salario

ORDER BY

e.monthly_income **DESC**;

- **Objetivo:** Ver empleados con mayores ingresos.
- **Valor:** Identificar top earners para análisis de equidad salarial.



The screenshot shows a database query result in a table viewer. The query is: `SELECT e.employee_number, e.monthly_income FROM employee_master_data AS e ORDER BY e.monthly_income DESC;` The table has two columns: 'employee_number' and 'monthly_income'. The results are ordered by monthly income in descending order. The first row shows employee 239 with a monthly income of 19,999. The last row shown is employee 1,625 with a monthly income of 19,658. The status bar at the bottom indicates '1470 row(s) fetched - 0,004s (0,001s fetch), on 2026-04-02 at 18:00:06'.

employee_number	monthly_income
239	19,999
1,035	19,973
1,191	19,943
226	19,926
787	19,859
1,282	19,847
1,038	19,845
1,740	19,833
1,255	19,740
1,338	19,717
1,423	19,701
1,867	19,665
1,625	19,658

- **Caso 2: Ordenar por antigüedad**

```
/*****
*****
Día 2 - 120    Diccionario Técnico Operativo    Caso 02: ORDER BY
*****
*****/
```

```
/*****
*****
Título: Análisis de Antigüedad de Empleados
```

Objetivo: Identificar y ordenar a los empleados según su tiempo de permanencia en la empresa para visualizar quiénes son los más recientes.

Descripción: Este script consulta la base de datos maestra de empleados para extraer el número de identificación y los años de servicio, ordenando los resultados de forma ascendente (de menor a mayor antigüedad).

Resultado CSV: day02_empl_years_at_company_ranking.csv

```
*****
*****/
```

-- Selecciona las columnas necesarias: identificación del empleado y años en la empresa

SELECT

e.employee_number,
e.years_at_company

-- Desde la tabla principal de datos maestros de empleados

FROM

employee_master_data AS e

-- Ordena los resultados por años de antigüedad (ascendente: menor a mayor)

ORDER BY

e.years_at_company ASC;

- **Objetivo:** Detectar empleados recién incorporados.
- **Valor:** Evaluar onboarding y retención inicial.

	employee_number	years_at_company
1	1.758	0
2	4	0
3	30	0
4	1.515	0
5	1.935	0
6	1.212	0
7	1.860	0
8	1.624	0
9	1.306	0
10	467	0
11	545	0
12	1.012	0
13	473	0

○ Caso 3: Ordenar por satisfacción laboral

```

/*****
*****
Día 2 - 120    Diccionario Técnico Operativo    Caso 02: ORDER BY
*****
*****/

```

```

/*****
*****
Título: Análisis de Satisfacción Laboral

```

Objetivo: Identificar a los empleados con mayores niveles de satisfacción laboral.

Descripción: Este script consulta la base de datos maestra de empleados para extraer el número de empleado y su puntuación de satisfacción, ordenándolos de mayor a menor para destacar a los más satisfechos.

Resultado CSV: day02_empl_job_satisfaction_ranking.csv

```

*****
*****/

```

-- Selecciona las columnas necesarias: número de empleado y nivel de satisfacción

```

SELECT
    e.employee_number,
    e.job_satisfaction
FROM
    employee_master_data AS e-- Indica la tabla de origen con los
    datos maestros
ORDER BY
    e.job_satisfaction DESC!-- Ordena los resultados de manera
    descendente (mayor a menor)

```

- **Objetivo:** Ver empleados más satisfechos.

- **Valor:** Identificar prácticas positivas a replicar.

	employee_number	e.job_satisfaction	job_satisfaction
1	184		4
2	1,853		4
3	192		4
4	621		4
5	622		4
6	623		4
7	2,012		4
8	1,240		4
9	1,240		4
10	1,238		4
11	62		4
12	632		4
13	195		4



PRIMARY KEY (PK):

- Identificador **único** por **fila**.
- Sintaxis:

`employee_number INT PRIMARY KEY`

- **Pitfall:** duplicar valores rompe integridad.
- **Caso 1:** Validar duplicados en `employee_number`

```

/*****
Día 2 - 120    Diccionario Técnico Operativo    Caso 01: PRIMARY KEY
(PK)
*****/

```

```

/*****
Título: Detección de Empleados Duplicados

```

Objetivo: Identificar registros duplicados en la tabla maestra de empleados.

Descripción: Este script escanea la tabla 'employee_master_data' para encontrar números de empleado que aparecen más de una vez, lo que indica una posible duplicidad de datos que debe ser corregida.

Resultado CSV: day02_duplicate_empl_records_audit.csv

```

*****/

```

-- Seleccionamos el número de empleado y contamos cuántas veces aparece cada uno.

```

SELECT
    e.employee_number,
    COUNT(*) AS total_registros

```

FROM

employee_master_data AS e-- Buscamos en la tabla maestra de empleados.

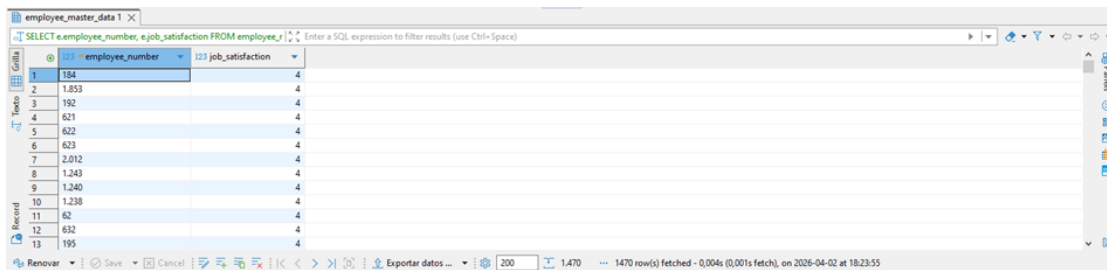
GROUP BY

e.employee_number-- Agrupamos los datos por cada número de empleado único.

HAVING

COUNT(*) > 1;;-- Filtramos para mostrar solo los grupos con más de un registro.

- **Objetivo:** Confirmar unicidad de PK.
- **Valor:** Garantizar integridad de registros.



	employee_number	job_satisfaction
1	184	4
2	1.853	4
3	192	4
4	621	4
5	622	4
6	623	4
7	2.012	4
8	1.243	4
9	1.240	4
10	1.238	4
11	62	4
12	632	4
13	195	4

○ Caso 2: Crear tabla con PK explícito

/*****

Título: Creación de Tabla de Empleados

Objetivo: Establecer una estructura base para almacenar información clave de los empleados.

Descripción: Este script crea una tabla denominada 'employees' para registrar el número de empleado, edad e ingresos mensuales, asegurando que cada empleado sea único.

*****/

-- Creación de la tabla 'employees' para almacenar los datos del personal

CREATE TABLE employees (

-- Número de empleado: ID único y clave primaria para identificar a cada trabajador

employee_number INT PRIMARY KEY,

-- Edad del empleado: Almacena la edad como número entero

age INT,

```
-- Ingresos mensuales: Almacena el sueldo mensual en números
enteros
monthly_income INT
);
```

- **Objetivo:** Definir PK en tabla nueva.
- **Valor:** Evitar duplicados desde el diseño.

○ Caso 3: Insertar registro único

```
/*****
*****/
```

Título: Inserción de Nuevo Empleado

Objetivo: Registrar un nuevo empleado en la tabla 'employees'.

Descripción: Inserta un registro específico con número de empleado, edad e ingresos mensuales en la base de datos para actualizar la nómina.

```
*****/
```

-- Bloque de inserción: Añade un nuevo registro a la tabla de empleados

```
INSERT IN TO employees (
    employee_number, -- Columna: Identificador único del
    empleado
    age, -- Columna: Edad del empleado
    monthly_income -- Columna: Salario mensual registrado
)
VALUES (
    1001, -- Valor: Número de empleado 1001
    35, -- Valor: 35 años de edad
    6000 -- Valor: 6000 unidades monetarias
);
```

- **Objetivo:** Insertar empleado con PK único.
- **Pitfall:** Si el número ya existe, falla.

FOREIGN KEY (FK) :

- Relación con otra **tabla**.
- **Sintaxis:**

**department_id INT REFERENCES
department_catalog(department_id)**

- **Pitfall:** insertar valor inexistente en catálogo.
- **Caso 1: Crear relación con departamentos**

```
/*****  
*****/
```

Título: Actualización de Estructura de Empleados

Objetivo: Vincular la tabla de empleados con la tabla de departamentos.

Descripción: Se añade una columna de clave foránea (foreign key) a la tabla

'employee_final' para referenciar el departamento al que pertenecen, asegurando la integridad referencial con 'department_catalog'.

```
*****/
```

-- 1. Modificar la tabla de empleados

ALTER TABLE employee_final

-- 2. Añadir nueva columna para el ID del departamento

ADD COLUMN department_id INT

-- 3. Establecer la restricción de llave foránea hacia el catálogo

REFERENCES department_catalog (department_id);

- **Objetivo:** Vincular empleados con catálogo de departamentos.
- **Valor:** Evitar inconsistencias en nombres de áreas.

- **Caso 2: Validar integridad referencial**

```
/*****  
*****/
```

Título: Identificación de Empleados con Departamentos Inexistentes

Objetivo: Detectar empleados asignados a departamentos que no existen en el catálogo maestro.

Descripción: Este script selecciona el número de empleado y su ID de departamento para aquellos registros cuyo 'department_id' no tiene un correspondiente en la

tabla 'department_catalog'. Ayuda a identificar errores de integridad referencial.

*****/

-- Inicia la selección de datos para el reporte

SELECT

e.employee_number, -- Muestra el número único del empleado

e.department_id-- Muestra el ID del departamento asignado

actualmente

FROM

employee_final AS e-- Tabla principal de empleados, definida con el alias 'e'

WHERE

-- 1. Asegura que el empleado tenga un departamento asignado antes de verificar

e.department_id IS NOT NULL

-- 2. Filtra registros donde el departamento no encuentra coincidencia (integridad referencial rota)

AND NOT EXISTS (

-- Subconsulta para revisar la tabla maestra de departamentos

SELECT 1

FROM department_catalog AS dc-- Tabla maestra, definida con alias 'd'

WHERE dc.department_id = e.department_id-- Compara el ID de empleado con el catálogo

);

- **Objetivo:** Detectar FKs inválidos.
- **Valor:** Garantizar consistencia.

○ **Caso 3: Insertar empleado con FK válido**

/*****

Título: Inserción de Nuevo Empleado

Objetivo: Registrar un nuevo empleado en la base de datos.

Descripción: Añade un registro individual a la tabla 'employee_final' con los datos

básicos de identificación, edad, ingresos y departamento.

```
*****
*****/
```

-- Bloque: Inserción de datos en la tabla de empleados
-- Descripción: Se inserta un registro nuevo con: número de empleado, edad, ingreso mensual y ID de departamento.

INSERT INTO

employee_final (**employee_number**, **age**, **monthly_income**,
department_id)

VALUES

(**1002**, **40**, **7000**, **3**);-- Define los valores: 1002 (ID), 40 (Edad), 7000 (Sueldo), 3 (Dpto)

- **Objetivo:** Insertar empleado en departamento existente.
- **Pitfall:** Si department_id=3 no existe, falla.



COPY:

- Carga masiva de **datos** desde **CSV**.
- **Sintaxis:**

COPY tabla

FROM 'ruta.csv' DELIMITER ',' CSV HEADER;

- **Pitfall:** archivo abierto en Excel bloquea carga.
- **Caso 1: Importar dataset IBM HR**

```
/*****
*****/
```

Título: Carga de Datos Maestros de Empleados (RRHH)

Objetivo: Importar información cruda de empleados desde un archivo CSV a la base de datos.

Descripción: Este script carga los datos del archivo 'WA_Fn-UseC_-HR-Employee-Attrition.csv' a la tabla 'employee_master_data' y actualiza las estadísticas de la tabla para optimizar futuras consultas.

```
*****
*****/
```

-- Definición de la ruta base donde se encuentra el archivo CSV.

```

@set carpeta_origen =
'C:/DM_Lab/DM_Roadmap_P1_120D/02_data/raw/rrhh/'

-- BLOQUE 1: Carga masiva de datos (COPY)
-- Lee el archivo CSV y lo inserta en la tabla especificada.
COPY employee_master_data
FROM '${carpeta_origen}WA_Fn-UseC_-HR-Employee-Attrition.csv'
WITH (
    FORMAT CSV,      -- Especifica que el archivo es de tipo texto plano
                      con comas.
    HEADER,          -- Indica que la primera fila del CSV son títulos y
                      debe saltarse.
    DELIMITER ',',   -- Define la coma como separador de columnas.
    ENCODING 'UTF8', -- Garantiza la correcta lectura de tildes y
                      caracteres especiales.
    FREEZE TRUE      -- Optimización: omite la creación de versiones de
                      filas (MVCC),
                      -- ideal para tablas nuevas o vacías, agilizando la
                      carga.
);

```

- **Objetivo:** Cargar dataset completo.
- **Pitfall:** Archivo abierto en Excel bloquea carga.

○ Caso 2: Exportar resultados a CSV

```

/*****
*****
Título: Exportación de Empleados con Rotación (Attrition)

Objetivo: Generar un archivo CSV con el detalle de todos los empleados
que han salido
de la empresa (rotación activa) para su análisis posterior.

Descripción: Este script selecciona los registros de la tabla
'employee_final' donde la
columna 'attrition' es 'Yes' y exporta los datos a un archivo físico local
en formato
CSV, garantizando la correcta codificación de caracteres.

*****/

```

```

-- 1. CONFIGURACIÓN DE RUTA

```

```
-- Definimos una variable para la ruta de la carpeta (DBeaver la
sustituirá
-- antes de enviar al servidor) para facilitar el cambio de ubicación en el
futuro.
```

```
@set carpeta_destino =
'C:/DM_Lab/DM_Roadmap_P1_120D/02_data/processed/rrhh/'
```

```
-- 2. OPERACIÓN DE EXPORTACIÓN (COPY)
```

```
-- Iniciamos la operación para extraer datos desde la base de datos a
un archivo local.
```

```
COPY (
```

```
-- Seleccionamos todas las columnas de la tabla final de
empleados.
```

```
SELECT *
```

```
FROM employee_final AS e
```

```
-- Filtramos únicamente aquellos registros donde la rotación
(attrition) es 'Yes'.
```

```
WHERE attrition = 'Yes'
```

```
)
```

```
-- Definimos la ruta completa y el nombre del archivo de salida
usando la variable definida antes.
```

```
TO '${carpeta_destino}employees_attrition.csv'
```

```
-- Especificamos el formato del archivo: CSV, con encabezados y
separado por comas.
```

```
WITH (
```

```
FORMAT CSV,
```

```
HEADER, -- Incluye los nombres de columna en la
primera línea.
```

```
DELIMITER ',', -- Separa los campos por comas.
```

```
ENCODING 'UTF8', -- Garantiza la correcta visualización de
caracteres especiales.
```

```
NULL 'NULL'-- Define que los valores nulos se escriban como
texto 'NULL'.
```

```
);
```

- **Objetivo:** Exportar empleados que abandonaron.
- **Valor:** Dataset listo para análisis en Python/Power BI.

Desglose de Lógica (Code Blueprint)

- **Por qué normalizar:**
 - Evita **duplicados** (ej. "Sales" repetido cientos de veces).
 - Facilita **mantenimiento** (renombrar un **departamento** = actualizar una **fila**).
- **Mindset SQL:**
 - **Claridad > complejidad:** queries legibles, nombres en **snake_case**.
 - **Optimización:** usar **índices** en PK/FK, evitar **SELECT *** en producción.
- **Clean Code:**
 - **SQL:** comentarios claros, bloques lógicos separados.
 - **Python:** PEP8, nombres **descriptivos**, modularidad.

Micro-Proyecto (Hands-on Workflow)

SQL:

Paso 1. Preparación del Entorno (Día 1)

- **Base de datos:** Crear `ibm_hr_analytics` en [PostgreSQL](#).
- **Dataset:** `WA_Fn-UseC_-HR-Employee-Attrition.csv`.
- **Herramientas:** PostgreSQL + DBeaver, Python (Pandas), Power BI

Paso 2. Identificación de Redundancia (Análisis)

- Observa la columna **Department** y **JobRole**. El nombre "Sales" se repite cientos de veces. Si mañana la empresa decide llamar a "Sales" como "Commercial", tendrías que actualizar miles de filas. **Solución: Crear tablas maestras.**

Paso 3. Creación de catálogos y tabla final

- Las **Tablas Maestras** son fundamentales en bases de datos relacionales para almacenar datos estáticos (ej. clientes, productos, usuarios). Eliminan redundancia y aseguran integridad referencial, evitando repetir información en tablas transaccionales.

```
/*  
*****  
*****
```

Título: Script de Creación de Base de Datos para Recursos Humanos

Objetivo: Crear un modelo de datos normalizado para almacenar información de empleados.

Descripción: Este script define catálogos para estandarizar datos (departamentos, roles, educación) y una tabla central para los datos específicos de cada empleado, asegurando la integridad referencial.

```
*****
*****/
```

```
--
=====
=====
-- BLOQUE 1: Creación de Catálogos (Tablas Maestras)
-- Objetivo: Almacenar valores únicos para evitar duplicidad y
estandarizar datos.
--
=====
=====
```

```
-- Catálogo de Departamentos
CREATE TABLE department_catalog (
    department_id SERIAL PRIMARY KEY, -- ID único
    autoincremental
    department_name VARCHAR (50) UNIQUE NOT NULL--
    Nombre del departamento (no nulo, único)
);
```

▪ Resultado PNG: day02_create_mstr_dept_catalog.png

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,048s
Start time	Sun Mar 29 19:14:18 CST 2026
Finish time	Sun Mar 29 19:14:18 CST 2026
Query	-- Bloque de creación de la tabla
	CREATE TABLE department_catalog (
	-- Define el ID del departamento como clave primaria, autoincrementable.
	department_id SERIAL PRIMARY KEY,
	-- Define el nombre del departamento: texto hasta 50 caracteres,
	-- obligatorio (NOT NULL) y sin repetirse (UNIQUE).
	department_name VARCHAR (50) UNIQUE NOT NULL
	)

```
-- Catálogo de Roles
CREATE TABLE job_role_catalog (
    job_role_id SERIAL PRIMARY KEY, -- ID único
    autoincremental
    job_role_name VARCHAR (50) UNIQUE NOT NULL-- Nombre
    del puesto (no nulo, único)
);
```

- **Resultado PNG: day02_create_mstr_job_role_ctlg.png**

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,013s
Start time	Sun Mar 29 19:27:45 CST 2026
Finish time	Sun Mar 29 19:27:45 CST 2026
Query	-- Creación de la tabla que funcionará como catálogo maestro de roles
	CREATE TABLE job_role_catalog(
	-- Identificador único autoincremental para cada rol (llave primaria)
	job_role_id SERIAL PRIMARY KEY,
	-- Nombre del rol (ej. "Gerente", "Analista"), máximo 50 caracteres,
	-- no permite duplicados y es obligatorio
	job_role_name VARCHAR (50) UNIQUE NOT NULL
	);

-- Catálogo de Campos Educativos

```
CREATE TABLE education_field_catalog (
    education_field_id SERIAL PRIMARY KEY, -- ID único
    autoincremental
    education_field_name VARCHAR (50) UNIQUE NOT NULL --
    Área de estudio (no nulo, único)
);
```

- **Resultado PNG: day02_create_mstr_education_field_ctlg.png**

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,016s
Start time	Sun Mar 29 19:32:39 CST 2026
Finish time	Sun Mar 29 19:32:39 CST 2026
Query	-- Se crea la tabla 'education_field_catalog' para almacenar el catálogo de campos.
	CREATE TABLE education_field_catalog(
	-- Define la clave primaria. 'SERIAL' crea un número entero autoincremental único.
	education_field_id SERIAL PRIMARY KEY,
	-- Define el nombre del campo educativo.
	-- VARCHAR(50): Permite texto de hasta 50 caracteres.
	-- UNIQUE: Asegura que no haya dos campos educativos con el mismo nombre.
	-- NOT NULL: Obliga a que el campo siempre tenga un valor (no permite nulos).



Paso 4. Poblar catálogos con valores únicos

```
/*****
*****/
```

Título: Migración de Catálogos desde Datos Maestros

Objetivo: Poblar las tablas de catálogo (Departamentos, Roles, Educación) a partir de la tabla maestra de empleados, asegurando la unicidad y limpiando valores nulos o vacíos.

Descripción: Este script lee la tabla 'employee_master_data' y extrae los valores

únicos de campos clave para alimentar tablas maestras normalizadas, evitando duplicados mediante comprobaciones 'NOT EXISTS'.

*****/

--

=====

=====

-- Bloque 1: Poblar el catálogo de departamentos

--

=====

=====

INSERT INTO department_catalog (department_name)
SELECT DISTINCT e.department-- Selecciona departamentos
únicos

FROM employee_master_data **AS** e

WHERE e.department **IS NOT NULL**-- Ignora valores nulos

AND e.department <> ''-- Ignora valores vacíos

-- Evita intentar insertar lo que ya existe en el catálogo

AND NOT EXISTS (

SELECT 1

FROM department_catalog **AS** dc

WHERE dc.department_name = e.department

);

▪ Resultado PNG: day02_insert_mstr_dept_catalog.png

Estadísticas 1	
Name	Value
Updated Rows	3
Execute time	0,018s
Start time	Sun Mar 29 19:39:30 CST 2026
Finish time	Sun Mar 29 19:39:30 CST 2026
Query	-- Bloque 1: Poblar el catálogo de departamentos
	-- Extrae nombres de departamentos únicos de la tabla maestra y los inserta en 'department_catalog'.
	INSERT INTO department_catalog (department_name)
	SELECT DISTINCT department-- Selecciona departamentos distintos
	FROM employee_master_data-- Desde la tabla origen
	WHERE department IS NOT NULL-- Filtra nulos
	AND department <> ''

--

=====

=====

-- Bloque 2: Poblar el catálogo de roles laborales (puestos)

--

=====

=====

INSERT INTO job_role_catalog (job_role_name)

```

SELECT DISTINCT e.job_role-- Selecciona puestos de trabajo
únicos
FROM employee_master_data AS e
WHERE e.job_role IS NOT NULL-- Ignora valores nulos
AND e.job_role <> ""-- Ignora valores vacíos
-- Asegura no duplicar roles ya existentes
AND NOT EXISTS (
    SELECT 1
    FROM job_role_catalog AS jrc
    WHERE jrc.job_role_name = e.job_role
);

```

- Resultado PNG: day02_insert_mstr_job_role_ctlg.png

Estadísticas 1	
Name	Value
Updated Rows	9
Execute time	0,006s
Start time	Sun Mar 29 19:43:54 CST 2026
Finish time	Sun Mar 29 19:43:54 CST 2026
Query	-- Bloque 2: Poblar el catálogo de roles laborales (puestos)
	-- Extrae los roles únicos de la tabla maestra para alimentar la tabla 'job_role_catalog'.
	INSERT INTO job_role_catalog (job_role_name)
	SELECT DISTINCT job_role-- Selecciona puestos únicos
	FROM employee_master_data-- Desde la tabla origen
	WHERE job_role IS NOT NULL-- Filtra nulos
	AND job_role <> ""

```

--
=====
-- Bloque 3: Poblar el catálogo de áreas educativas
--
=====
=====
INSERT INTO education_field_catalog (education_field_name)
SELECT DISTINCT e.education_field-- Selecciona áreas de estudio
únicas
FROM employee_master_data AS e
WHERE e.education_field IS NOT NULL-- Ignora valores nulos
AND e.education_field <> ""-- Ignora valores vacíos
-- Asegura no duplicar áreas de estudio ya existentes
AND NOT EXISTS (
    SELECT 1
    FROM education_field_catalog AS efc
    WHERE efc.education_field_name = e.education_field
);

```

▪ **Resultado PNG: day02_insert_mstr_eduction_field_ctlg.png**

Estadísticas 1	
Name	Value
Updated Rows	6
Execute time	0,006s
Start time	Sun Mar 29 19:49:15 CST 2026
Finish time	Sun Mar 29 19:49:15 CST 2026
Query	-- Bloque 3: Poblare el catálogo de áreas educativas -- Extrae los campos de estudio únicos de la tabla maestra para la tabla 'education_field_catalog'. INSERT INTO education_field_catalog(education_field_name) SELECT DISTINCT education_field-- Selecciona áreas educativas únicas FROM employee_master_data-- Desde la tabla origen WHERE education_field IS NOT NULL-- Filtra nulos AND education_field <> ''
CST es Editable Inserción inteligente 93 : 10 : 4628 Set: 0 0 :	

 Paso 5. Validación de Catálogos Maestros de RRHH:

/*****

Título: Validación de Catálogos Maestros de RRHH

Objetivo: Verificar la correcta carga de datos en los catálogos fundamentales antes de iniciar procesos de importación de empleados o reportes.

Descripción: Este script ejecuta tres consultas independientes para listar y ordenar los catálogos de departamentos, roles de trabajo y campos educativos, permitiendo al usuario asegurar la integridad de los datos referenciales (catálogos).

*****/

-- BLOQUE 1: Validación de Departamentos

-- Propósito: Visualizar la lista de departamentos activos para confirmar su existencia y correcta ortografía.

SELECT *

FROM department_catalog **AS** dc

ORDER BY dc.department_name;-- Ordena alfabéticamente para facilitar la lectura.

- **Resultado PNG: day02_mstr_dept_catalog_data.png**

dept_id	dept_name
1	Human Resources
2	Research & Development
3	Sales

-- BLOQUE 2: Validación de Roles de Trabajo
 -- Propósito: Revisar el catálogo de puestos/roles para asegurar que las denominaciones sean correctas.

SELECT *
FROM job_role_catalog **AS** jrc
ORDER BY jrc.job_role_name;-- Ordena alfabéticamente por nombre del puesto.

- **Resultado PNG: day02_mstr_job_role_ctlg_data.png**

job_role_id	job_role_name
3	Healthcare Representative
4	Human Resources
5	Laboratory Technician
1	Manager
6	Manufacturing Director
9	Research Director
2	Research Scientist
8	Sales Executive
7	Sales Representative

-- BLOQUE 3: Validación de Campos Educativos
 -- Propósito: Confirmar la lista de áreas de estudio o campos educativos disponibles.

SELECT *
FROM education_field_catalog **AS** efc
ORDER BY efc.education_field_name;-- Ordena alfabéticamente por nombre del campo educativo.

- **Resultado PNG: day02_mstr_education_field_ctlg_data.png**

education_field_id	education_field_name
2	Human Resources
4	Life Sciences
1	Marketing
6	Medical
3	Other
5	Technical Degree

Paso 6. Creación de Tabla Maestra de Empleados Final

/*****

*****/

Título: Preparación y Enriquecimiento de Datos de Empleados

Objetivo: Crear una tabla de trabajo definitiva y vincularla con catálogos.

Descripción: Este script realiza dos acciones principales:

1. Crea una copia de seguridad/trabajo de los datos maestros de empleados.

2. Añade nuevas columnas para vincular (hacer relaciones) al empleado con tablas maestras de departamentos, puestos y áreas de estudio, garantizando la integridad de los datos mediante restricciones (Foreign Keys).

*****/

*****/

--

=====

=====

-- 1. CREACIÓN DE LA TABLA DE TRABAJO

--

=====

=====

-- Se usa DROP si existe para asegurar un estado limpio en lugar de IF NOT EXISTS,

-- lo cual es más seguro si la estructura del origen cambió.

DROP TABLE IF EXISTS employee_final;

-- Se utiliza CTAS (Create Table As Select) que es más rápido que INSERT.

-- Si la tabla es muy grande, añadir 'WITH DATA' o 'NOLOGGING' (en Oracle)

-- puede acelerarlo, aunque depende del motor.

CREATE TABLE employee_final AS

SELECT *

FROM employee_master_data;-- Copia toda la estructura y datos de la tabla maestra.

▪ Resultado PNG: day02_create_tbl_empl_fnl.png

Estadísticas 1	
Name	Value
Updated Rows	1470
Execute time	0,016s
Start time	Sun Mar 29 20:43:24 CST 2026
Finish time	Sun Mar 29 20:43:24 CST 2026
Query	<pre>-- ===== -- 1. CREACIÓN DE LA TABLA DE TRABAJO ===== -- Se crea la tabla 'employee_final' basándose en 'employee_master_data'. -- 'IF NOT EXISTS' evita errores si el script se ejecuta más de una vez. CREATE TABLE IF NOT EXISTS employee_final AS SELECT * FROM employee_master_data</pre>
CST	es Editable
Inserción inteligente	19 : 7 : 1080
Set: 0 0	:

```
--
=====
=====
-- 2. ENRIQUECIMIENTO Y RELACIONES (ALTER TABLE)
--
=====
=====
-- Modificamos la tabla 'employee_final' para añadir columnas de
relación.
```

ALTER TABLE employee_final

-- Añade columna para el departamento y lo vincula con el catálogo de departamentos.

ADD COLUMN department_id INT REFERENCES
department_catalog (department_id),

-- Añade columna para el puesto y lo vincula con el catálogo de roles/puestos.

ADD COLUMN job_role_id INT REFERENCES job_role_catalog
(job_role_id),

-- Añade columna para el área educativa vinculada al catálogo de estudios.

ADD COLUMN education_field_id INT REFERENCES
education_field_catalog (education_field_id);

▪ Resultado PNG: day02_alter_tbl_empl_fnl.png

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,015s
Start time	Sun Mar 29 20:52:55 CST 2026
Finish time	Sun Mar 29 20:52:55 CST 2026
Query	<pre>-- ===== -- 2. ENRIQUECIMIENTO Y RELACIONES (ALTER TABLE) ===== -- Modificamos la tabla 'employee_final' para añadir columnas de relación. ALTER TABLE employee_final -- Añade columna para el departamento y lo vincula con el catálogo de departamentos. ADD COLUMN department_id INT REFERENCES department_catalog(department_id), -- Añade columna para el puesto y lo vincula con el catálogo de roles/puestos.</pre>
CST	es Editable
Inserción inteligente	29 : 12 : 1750
Set: 0 0	:

Paso 7. Insertar datos en tabla final con PK y FK

/*****

*****/

Título: Actualización de llaves foráneas en tabla de empleados

Objetivo: Normalizar la tabla 'employee_final' sustituyendo nombres de texto por sus correspondientes IDs (llaves primarias) de tablas de catálogo.

Descripción: El script toma los valores de texto (departamento, puesto, campo de estudio) de la tabla 'employee_final', los busca en sus respectivos catálogos y actualiza la tabla principal con el ID numérico correspondiente para mejorar la integridad referencial.

*****/

*****/

-- Actualizar la tabla final de empleados con los IDs correctos de los catálogos

UPDATE employee_final **AS** e
SET

-- Asigna el ID del departamento buscando por nombre

department_id = dc.department_id,

-- Asigna el ID del puesto de trabajo buscando por nombre

job_role_id = jrc.job_role_id,

-- Asigna el ID del campo educativo buscando por nombre

education_field_id = efc.education_field_id

-- Define las tablas de catálogo que actúan como fuente de búsqueda

FROM

department_catalog **AS** dc,

job_role_catalog **AS** jrc,

education_field_catalog **AS** efc

-- Relaciona las tablas de catálogo con los textos descriptivos de la tabla de empleados

WHERE e.department = dc.department_name-- Coincidencia de departamento

AND e.job_role = jrc.job_role_name-- Coincidencia de puesto (rol)

AND e.education_field = efc.education_field_name;--

Coincidencia de campo de estudio

- Resultado PNG: day02_update_tbl_empl_fnl.png

Estadísticas 1 X	
Name	Value
Updated Rows	1470
Execute time	0,36s
Start time	Sun Mar 29 23:25:35 CST 2026
Finish time	Sun Mar 29 23:25:35 CST 2026
Query	<pre>-- Actualizar la tabla final de empleados con los IDs correctos de los catálogos UPDATE employee_final e SET -- Asigna el ID del departamento buscando por nombre department_id = d.department_id, -- Asigna el ID del puesto de trabajo buscando por nombre job_role_id = jr.job_role_id, -- Asigna el ID del campo educativo buscando por nombre</pre>

Paso 8. Limpieza de la tabla final de empleados

/*****

Título: Limpieza de la tabla final de empleados

Objetivo: Optimizar la estructura de la tabla 'employee_final' eliminando columnas de texto redundantes que ya han sido procesadas o mapeadas a valores numéricos/categorías, mejorando el rendimiento y reduciendo el espacio de almacenamiento.

Descripción: Se eliminan las columnas 'department', 'job_role' y 'education_field' debido a que la información ya se encuentra normalizada en otras tablas o columnas dentro de la base de datos.

*****/

ALTER TABLE employee_final

```
-- Elimina la columna 'department' si existe
```

DROP COLUMN IF EXISTS department,

```
-- Elimina la columna 'job_role' si existe
```

DROP COLUMN IF EXISTS job_role,

```
-- Elimina la columna 'education_field' si existe
```

DROP COLUMN IF EXISTS education_field;

▪ Resultado PNG: day02_drop_col_empl_fnl.png

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	0,007s
Start time	Sun Mar 29 23:32:57 CST 2026
Finish time	Sun Mar 29 23:32:57 CST 2026
Query	-- Eliminamos las columnas de texto originales
	-- Esta sentencia modifica la estructura de la tabla 'employee_final' para borrar
	-- los campos de texto que ya no son necesarios en el análisis.
	ALTER TABLE employee_final
	DROP COLUMN department, -- Elimina la columna con el nombre del departamento
	DROP COLUMN job_role, -- Elimina la columna con el nombre del puesto
	DROP COLUMN education_field

Paso 9. Validación de Estructura de Empleados

/*

=====

=====

Título: Validación de Estructura de Empleados

Objetivo: Verificar la correcta integración de datos.

Descripción: Consulta de muestra para validar que la tabla final de empleados esté correctamente vinculada con sus catálogos (departamento, puesto, educación), mostrando datos clave de rendimiento e ingresos.

=====

===== */

SELECT

-- Seleccionamos las columnas clave para el reporte

e.employee_number, -- ID único del empleado

dc.department_name, -- Nombre del departamento (traído del catálogo)

jrc.job_role_name, -- Nombre del puesto (traído del catálogo)

efc.education_field_name, -- Área de estudio (traída del catálogo)

e.monthly_income, -- Ingreso mensual actual

e.performance_rating -- Calificación de desempeño

FROM

-- Tabla base: Datos finales de los empleados

employee_final AS e

-- Unimos con catálogo de departamentos para obtener el nombre

JOIN department_catalog AS dc ON e.department_id = dc.department_id

-- Unimos con catálogo de puestos para obtener el nombre del rol

JOIN job_role_catalog AS jrc ON e.job_role_id = jrc.job_role_id

```
-- Unimos con catálogo de estudios para obtener el área educativa
JOIN education_field_catalog AS efc ON e.education_field_id =
efc.education_field_id
-- Ordenamos por número de empleado para garantizar un orden
consistente
ORDER BY
    e.employee_number
-- Limitamos el resultado a las primeras 10 filas para una vista
rápida
LIMIT 10;
```

▪ Resultado PNG: day02_empl_fnl_data_sample.png

employee_number	department_name	job_role_name	education_field_name	monthly_income	performance_rating
23	Sales	Manager	Life Sciences	15.427	3
1	Sales	Sales Executive	Life Sciences	5.993	3
2	Research & Development	Research Scientist	Life Sciences	5.130	4
4	Research & Development	Laboratory Technician	Other	2.090	3
5	Research & Development	Research Scientist	Life Sciences	2.909	3
7	Research & Development	Laboratory Technician	Medical	3.468	3
8	Research & Development	Laboratory Technician	Life Sciences	3.966	3
10	Research & Development	Laboratory Technician	Medical	2.670	4
11	Research & Development	Laboratory Technician	Life Sciences	2.693	4
12	Research & Development	Manufacturing Director	Life Sciences	9.526	4



Paso 10. Controles de la calidad de los datos (employee_final)

- Confirma que los datos se cargaron correctamente:

```
/*****
*****/
```

Título: Visualización Preliminar de Empleados

Objetivo: Obtener una muestra rápida de los datos almacenados en la tabla final de empleados.

Descripción: Este script selecciona todos los campos de la tabla 'employee_final' y limita el resultado a los primeros 10 registros para verificar la estructura y calidad de los datos sin cargar toda la tabla.

```
*****/
```

-- Selecciona todas las columnas (*) de la tabla principal de empleados

SELECT *

-- Indica la fuente de datos (tabla employee_final)

FROM employee_final

-- Limita la visualización solo a los primeros 10 registros

LIMIT 10;

■ **Resultado PNG: day02_empl_fnl_data_quality.png**

	age	gender	marital_status	over18	attrition	business_travel	distance_from_home	job_level	employee_number	employee_count	month
1	33	Female	Married	Y	No	Travel_Rarely	2	4	23	1	
2	41	Female	Single	Y	Yes	Travel_Rarely	1	2	1	1	
3	49	Male	Married	Y	No	Travel_Frequently	8	2	2	1	
4	37	Male	Single	Y	Yes	Travel_Rarely	2	1	4	1	
5	33	Female	Married	Y	No	Travel_Frequently	3	1	5	1	
6	27	Male	Married	Y	No	Travel_Rarely	2	1	7	1	
7	32	Male	Single	Y	No	Travel_Frequently	2	1	8	1	
8	59	Female	Married	Y	No	Travel_Rarely	3	1	10	1	
9	30	Male	Divorced	Y	No	Travel_Rarely	24	1	11	1	
10	38	Male	Single	Y	No	Travel_Frequently	23	3	12	1	

 Paso 11. Detección de registros duplicados:

```
/*  
*****  
*****
```

Título: Detección de Empleados Duplicados

Objetivo: Identificar inconsistencias en la base de datos de empleados.

Descripción: Este script analiza la tabla 'employee_final' para encontrar números de empleado que aparecen más de una vez, lo que indica registros duplicados que deben ser revisados.

```
*****  
*****/  
*****
```

SELECT

-- Selecciona el número de empleado para identificarlo
e.employee_number,

-- Cuenta cuántas veces aparece cada número de empleado

COUNT(*) AS repeticiones

-- Define la tabla de origen de los datos

FROM employee_final AS e

-- Agrupa los resultados por número de empleado para poder contarlos

GROUP BY e.employee_number

-- Filtra y muestra solo aquellos grupos con más de un registro (duplicados)

HAVING COUNT(*) > 1;

▪ Resultado PNG: day02_duplectae_fnl_records_audit.png

employee_number	COUNT(*) AS repeticiones

Paso 12. Identificación de Valores Nulos en Columnas Críticas:

```

/*****
****

```

Título: Consolidación y Validación de Empleados Activos

Objetivo: Obtener una vista completa de los empleados, uniéndolos información de departamentos y puestos, asegurando la calidad de los datos.

Descripción: Este script combina la tabla principal de empleados (employee_final) con los catálogos de departamentos y puestos para obtener nombres legibles en lugar de IDs. Además, filtra los registros para asegurar que la información tenga un mínimo de calidad (datos esenciales).

```

****
*****/

```

SELECT

```

    e.employee_number,    -- Número único de
                           identificación del empleado
    dc.department_name,    -- Nombre del departamento
                           (obtenido del catálogo)
    jrc.job_role_name,     -- Nombre del puesto de trabajo
                           (obtenido del catálogo)
    e.monthly_income-- Salario mensual del empleado

```

FROM employee_final **AS** e

-- Une la tabla de empleados con el catálogo de departamentos para obtener el nombre del área

```

JOIN department_catalog AS dc ON e.department_id =
dc.department_id

```

-- Une con el catálogo de puestos para obtener el nombre del rol laboral

JOIN job_role_catalog **AS** jrc **ON** e.job_role_id = jrc.job_role_id

-- CRITERIOS DE VALIDACIÓN: Filtra para mantener registros con información mínima necesaria

WHERE

-- 1. Verifica que el empleado tenga un número de identificación asignado

e.employee_number IS NOT NULL OR

-- 2. Verifica que el empleado esté vinculado a un departamento conocido

dc.department_name IS NOT NULL OR

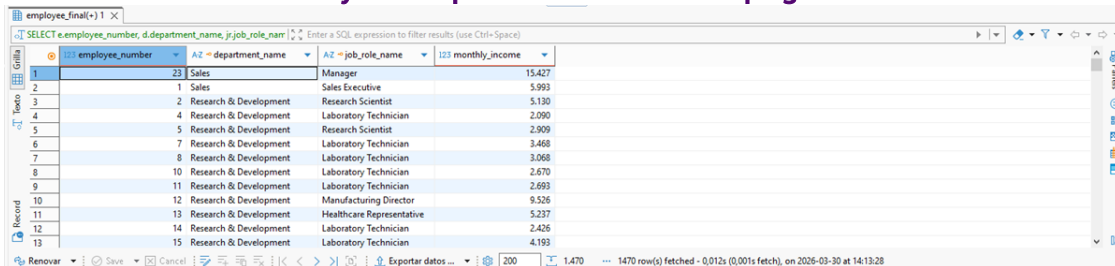
-- 3. Verifica que el empleado tenga un puesto de trabajo definido

jr.job_role_name IS NOT NULL OR

-- 4. Verifica que exista información sobre su salario mensual

e.monthly_income IS NOT NULL;

▪ Resultado PNG: day02_empl_fnl_data_cleaned.png



employee_number	department_name	job_role_name	monthly_income
23	Sales	Manager	15427
1	Sales	Sales Executive	5993
2	Research & Development	Research Scientist	5130
4	Research & Development	Laboratory Technician	2090
5	Research & Development	Research Scientist	2909
7	Research & Development	Laboratory Technician	3468
8	Research & Development	Laboratory Technician	3068
10	Research & Development	Laboratory Technician	2670
11	Research & Development	Laboratory Technician	2698
12	Research & Development	Manufacturing Director	9526
13	Research & Development	Healthcare Representative	5237
14	Research & Development	Laboratory Technician	2426
15	Research & Development	Laboratory Technician	4193

Paso 13. Auditoría de Integridad (Valores fuera de rango o vacíos):

/*****

*****/

Título: Identificación de Registros de Empleados con Datos Incompletos o Inválidos

Objetivo: Detectar empleados cuyo nombre de departamento no esté definido (vacío/nulo) o que tengan un ingreso mensual nulo o no positivo (≤ 0).

Descripción: Este script cruza la tabla de empleados con la de departamentos para obtener los nombres de los departamentos y filtra aquellos registros que presentan

inconsistencias en el nombre del departamento o en el monto de ingresos.

```
*****  
*****/
```

SELECT

e.employee_number, -- Selecciona el número
identificador único del empleado
dc.department_name, -- Selecciona el nombre del
departamento asociado
e.monthly_income-- Selecciona el ingreso mensual del
empleado

FROM

employee_final AS e-- Tabla principal de empleados
(alias 'e')

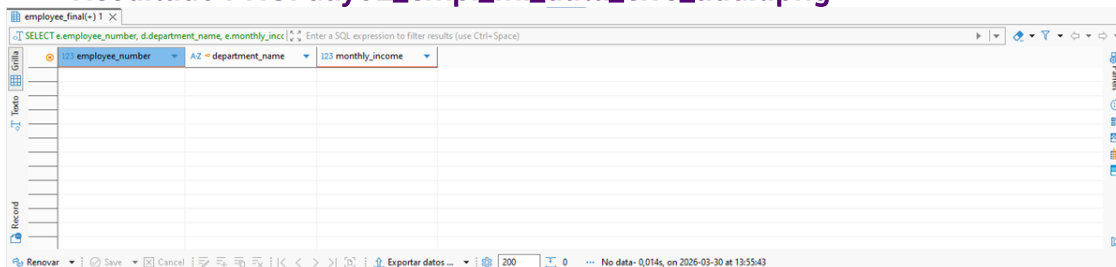
JOIN

department_catalog AS dc-- Une con la tabla catálogo
de departamentos (alias 'dc')
ON e.department_id = dc.department_id-- Cruza las
tablas usando el ID de departamento

WHERE

-- Filtro de Calidad de Datos:
-- 1. dc.department_name: Comprueba si es nulo, vacío
("") o espacios en blanco
-- 2. e.monthly_income: Comprueba si el ingreso es 0 o
menor (inválido)
(TRIM(COALESCE(dc.department_name, "")) = "
OR e.monthly_income <= 0);

▪ Resultado PNG: day02_empl_fnl_data_errs_audit.png



employee_number	department_name	monthly_income
103	AZ	103

Paso 14. Conteo total de colaboradores en la tabla principal:

```
/*****  
*****
```

Título: Conteo Total de Empleados

Objetivo: Obtener el número total de registros en la tabla de empleados.

Descripción: Este script realiza una consulta rápida para conocer el volumen actual de empleados censados en el sistema.

```
*****  
*****/
```

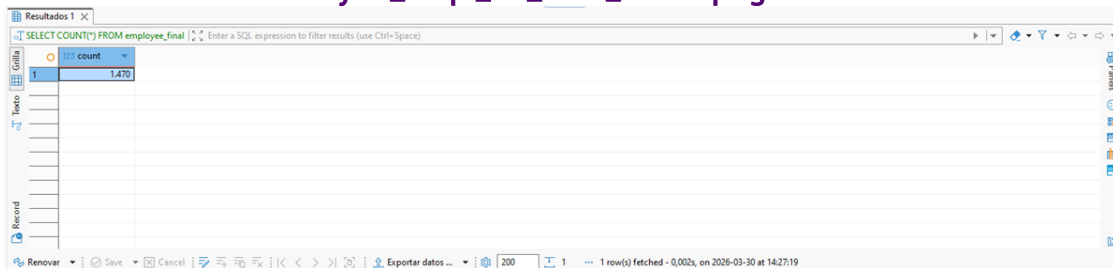
SELECT

COUNT(*) -- Cuenta el total de filas (registros) en la tabla, incluyendo nulos

FROM

employee_final;-- Especifica la tabla principal que contiene el censo de empleados

▪ Resultado PNG: day02_empl_fnl_data_count.png



The screenshot shows a database query result window titled 'Resultados 1'. The query is 'SELECT COUNT(*) FROM employee_final'. The result is displayed in a table with one row and one column, showing the value '1,470'. The window includes a toolbar with various icons and a status bar at the bottom indicating '1 row(s) fetched - 0,002s, on 2026-03-30 at 14:27:19'.

	count
1	1,470



Paso 15. Distribución de la plantilla por departamento:

```
/*****  
*****
```

Título: Reporte de Conteo de Empleados por Departamento

Objetivo: Obtener el número total de empleados que trabajan en cada área de la empresa.

Descripción: Este script consulta la tabla de empleados, la relaciona con el catálogo de departamentos para obtener el nombre real de cada área, y realiza un conteo agrupado para mostrar el total de personal por departamento.

```
*****  
*****/
```

SELECT

-- Selecciona el nombre del departamento de la tabla relacionada (d)

dc.department_name,

-- Cuenta el total de empleados (filas) agrupados por departamento

COUNT(*) AS total_empleados

-- Selecciona la tabla de empleados como fuente principal (alias e)

FROM employee_final AS e

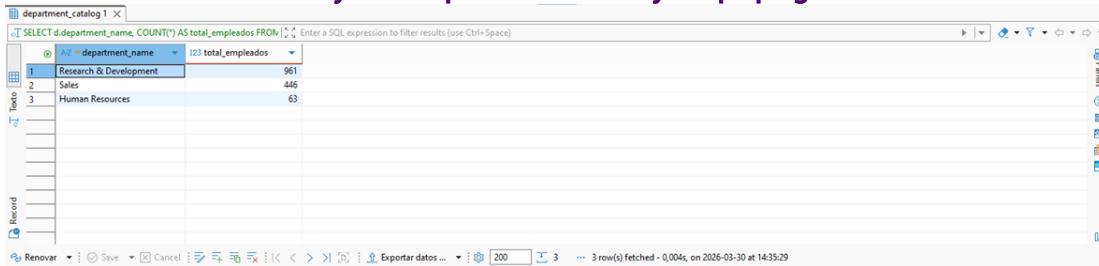
-- Une con el catálogo de departamentos (alias d) usando el ID compartido

JOIN department_catalog AS dc ON e.department_id = dc.department_id

-- Agrupa el conteo anterior según el nombre del departamento

GROUP BY dc.department_name;

▪ Resultado PNG: day02_empl_fnl_count_by_dept.png



	department_name	total_empleados
1	Research & Development	961
2	Sales	446
3	Human Resources	63



Paso 16. Queries integradoras (SELECT + WHERE + ORDER BY)

▪ Caso 1: Filtrar empleados con riesgo de rotación (WHERE + JOIN)

/*****
*****/

Título: Reporte de Empleados con Alto Ingreso y Baja Satisfacción.

Objetivo: Identificar empleados con salarios superiores a 5,000 que reportan una baja satisfacción laboral (menor a 2), para análisis de retención.

Descripción: Este script cruza datos de empleados con catálogos de departamentos y puestos, filtrando por ingresos y satisfacción, y ordenando de mayor a menor sueldo.

Resultado CSV: day02_hr_risk_analysis_results.csv

*****/

SELECT

e.employee_number, -- Número único del empleado
dc.department_name, -- Nombre del departamento al
que pertenece
jrc.job_role_name, -- Nombre del puesto o rol
laboral
e.monthly_income-- Sueldo mensual del empleado

FROM

employee_final AS e-- Tabla principal de empleados
(alias 'e')

JOIN

department_catalog AS dc-- Una tabla de
departamentos (alias 'd')
ON e.department_id = dc.department_id-- Conexión por
ID de departamento

JOIN

job_role_catalog AS jrc-- Una tabla de puestos (alias 'j')
ON e.job_role_id = jrc.job_role_id-- Conexión por ID de
puesto

WHERE

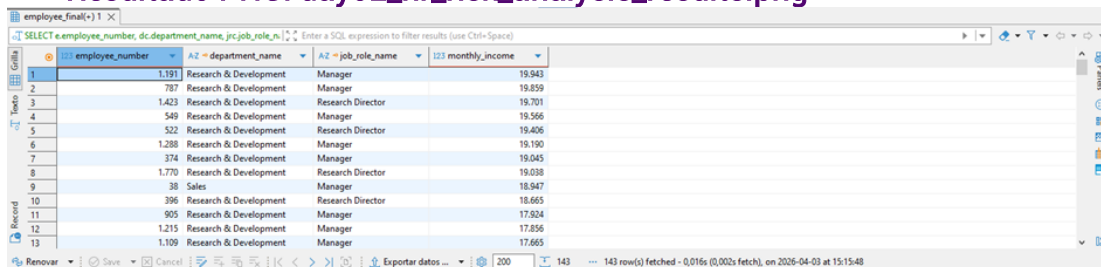
e.monthly_income > 5000-- Filtro 1: Empleados que
ganan más de 5,000
AND e.job_satisfaction < 2-- Filtro 2: Empleados con
bajo nivel de satisfacción (escala 1-4)

ORDER BY

e.monthly_income DESC-- Ordena el resultado del
sueldo más alto al más bajo

- **Objetivo:** Detectar empleados con alto salario y baja satisfacción.

Resultado PNG: day02_hr_risk_analysis_results.png



	employee_number	department_name	job_role_name	monthly_income
1	1191	Research & Development	Manager	19.943
2	787	Research & Development	Manager	19.859
3	1423	Research & Development	Research Director	19.701
4	549	Research & Development	Manager	19.566
5	522	Research & Development	Research Director	19.406
6	1288	Research & Development	Manager	19.190
7	374	Research & Development	Manager	19.045
8	1770	Research & Development	Research Director	19.038
9	38	Sales	Manager	18.947
10	396	Research & Development	Research Director	18.665
11	905	Research & Development	Manager	17.924
12	1215	Research & Development	Manager	17.856
13	1109	Research & Development	Manager	17.665

- **Insight:**

- **Desafío en la Cúpula:** Sueldos elite (\$19,943) presentan satisfacción nivel 1. El dinero no retiene, el problema es clima o retos, costoso de reemplazar.
- **Riesgo en Research & Development (R&D):** La mayoría de bajas/quejas provienen de **Research & Development (R&D)**. La innovación y proyectos estratégicos están en peligro por desmotivación. Requiere evaluar sobrecarga y visión.
- **Riesgo de "Caza de Talentos":** Los especialistas de alto rango (> \$5,000) son el **blanco principal** de la competencia. El descontento laboral de este **talento altamente empleable** facilita su **fuga**, generando una pérdida masiva de **capital intelectual**.
- **Inversión de Nómina Ineficiente:** Se asigna un alto flujo de caja a sueldos de empleados **descomprometidos**. Este **bajo rendimiento** implica que el negocio no aprovecha el **potencial humano**, desperdiciando recursos financieros.

▪ **Caso 2: Ranking de ingresos por departamento (ORDER BY)**

```

/*****
*****

```

Título: Reporte de Promedio de Ingresos por Departamento

Objetivo: Calcular y visualizar el salario mensual promedio de los empleados agrupados por departamento, ordenados del mayor al menor.

Descripción: Este script combina la información de empleados y departamentos, realiza el cálculo del promedio y ordena el resultado para identificar fácilmente los departamentos con mayor remuneración promedio.

Resultado CSV: day02_avg_income_by_dept.csv

```

*****
*****/

```

-- Selecciona el nombre del departamento y calcula el promedio de ingresos mensuales

SELECT

dc.department_name,

-- Calcula el promedio de ingresos (monthly_income) y lo redondea a 2 decimales para mejor lectura

ROUND(AVG(e.monthly_income),2) AS avg_income

-- Utiliza la tabla de empleados como fuente principal (alias 'e')

FROM

employee_final e

-- Une con la tabla de catálogo de departamentos (alias 'd') usando el ID compartido

JOIN

department_catalog AS dc

ON e.department_id = dc.department_id

-- Agrupa los resultados para que el promedio se calcule por cada departamento

GROUP BY

dc.department_name

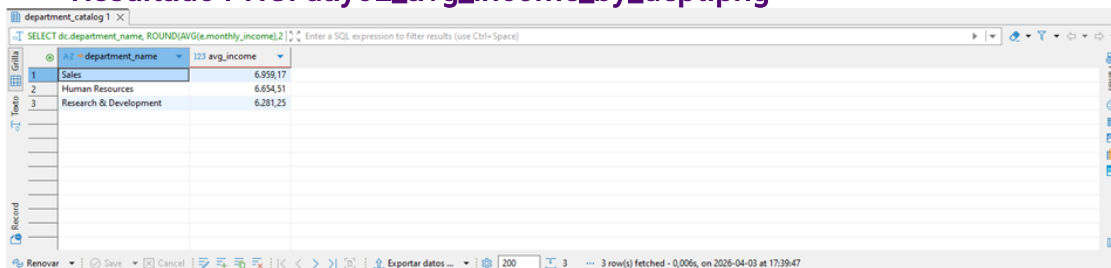
-- Ordena el reporte final de mayor a menor según el ingreso promedio calculado

ORDER BY

avg_income **DESC**;

- **Objetivo:** Ver departamentos con mayor promedio salarial.

▪ **Resultado PNG:** day02_avg_income_by_dept.png



The screenshot shows a database query result in a tool like DBeaver. The query is: `SELECT dc.department_name, ROUND(AVG(e.monthly_income),2)`. The results are displayed in a table with two columns: 'department_name' and 'avg_income'. The data is as follows:

department_name	avg_income
Sales	6.959,17
Human Resources	6.654,51
Research & Development	6.281,25

- **Insight:**
 - **El Departamento de Mayor Valor Percibido: Ventas** lidera con **\$6,959.17** promedio, reflejando una **estrategia de incentivos** agresivos para **atraer** y **retener** al equipo comercial, fundamental para la **generación de ingresos**.
 - **Soporte Estratégico de Alto Nivel: Human Resources** le sigue cerca con **\$6,654.51**, evidenciando que no es un área meramente administrativa, sino una unidad **estratégica** enfocada en la **gestión** y **retención** de **talento** especializado.
 - **R&D: Eficiencia en el Volumen: Research & Development (R&D)**, siendo el **corazón técnico**, presenta el **promedio salarial** más bajo (\$6,281.25). Esto se debe a su **estructura piramidal**, con gran base de **científicos junior**, logrando una **economía de escala** y **eficiencia en costos** operativos para la **innovación**.
 - **Equidad Interna:** La diferencia entre el máximo y mínimo salario es de apenas \$677.92 (~10%). Se observa una **consistencia salarial**

notable, lo que demuestra **salud organizacional** y **equidad interna**, evitando **brechas de desigualdad** y facilitando la **colaboración entre equipos**.

- **Resumen Estratégico:**
 - **Inversión** alineada a objetivos (**Sales, HR, R&D**) con **distribución equilibrada**.
- **Recomendación:**
 - Analizar la **desviación estándar salarial** para mitigar el alto riesgo de **rotación de personal** en la base, provocado por **desigualdad interna**, a pesar de los **promedios** parecidos.

▪ **Caso 3: Validación de PK (PRIMARY KEY)**

```
/*****  
*****/
```

Título: Detección de Registros Duplicados de Empleados

Objetivo: Identificar qué empleados aparecen más de una vez en la base de datos.

Descripción: Este script analiza la tabla 'employee_final' para encontrar números de empleado (employee_number) que están duplicados, lo cual puede indicar errores de carga o datos redundantes.

Resultado CSV: day02_duplicated_empl_records_cnt.csv

```
*****/
```

-- 1. Selecciona el número de empleado y cuenta cuántas veces aparece cada uno.

SELECT

e.employee_number,
COUNT(*) AS total_apariciones

FROM

employee_final AS e-- 2. Especifica la tabla maestra de empleados a consultar.

GROUP BY

e.employee_number-- 3. Agrupa los resultados por el número de empleado.

HAVING

COUNT(*) > 1;-- 4. Filtra para mostrar solo los empleados con más de un registro (duplicados).

- Resultado PNG: day02_duplicated_empl_records_cnt.png



Título: Exportación de Dataset de Empleados en Riesgo

Descripción: El script selecciona empleados con ingresos superiores a 5000 y satisfacción menor a 2, cruzando tablas de departamentos y roles para obtener datos legibles. El resultado se guarda en un CSV para análisis posterior en Python o Power BI.

***** /

- Seleccionamos las columnas necesarias para el análisis

e.employee_number, -- Número identificador único del empleado

dc.department_name, -- Nombre del departamento
(mapeado desde catálogo)

```
jrc.job_role_name,      -- Nombre del puesto específico
                        (mapeado desde catálogo)
```

```

        e.monthly_income,      -- Ingreso mensual actual para
                                evaluar costo-oportunidad
        e.job_satisfaction,    -- Nivel de satisfacción (1=Bajo,
                                4=Alto)
        e.attrition-- Indica si el empleado ya dejó la empresa
                                (Yes/No)
-- Definimos la tabla principal de empleados
FROM
    employee_final AS e
-- Unimos con catálogos para obtener nombres legibles en lugar
de IDs
JOIN
    department_catalog AS dc
    ON e.department_id = dc.department_id
JOIN
    job_role_catalog AS jrc
    ON e.job_role_id = jrc.job_role_id
-- Filtramos para enfocar en: Altos ingresos (>5000) Y baja
satisfacción (<2)
WHERE e.monthly_income > 5000
      AND e.job_satisfaction < 2
)
-- Definimos la ruta y nombre del archivo de salida
TO'C:/DM_Lab/DM_Roadmap_P1_120D/02_data/processed/rrhh/e
mployees_attrition.csv'
-- Configuramos el formato del archivo CSV
WITH(
    FORMAT CSV,      -- Formato estándar separado por
comas
    DELIMITER ',',   -- Coma como separador de columnas
    HEADER TRUE,     -- Incluye los nombres de las columnas
en la primera línea
    ENCODING 'UTF8'  -- Soporte para caracteres especiales
(acentos, ñ)
);

```

- **Objetivo:** Exportar dataset de empleados en riesgo.
- **Uso posterior:** Cargar en **Python** o **Power BI** para análisis.

- **Resultado PNG: day02_hr_attrition_risk_factors_exp_data.png**

Name	Value
Updated Rows	143
Execute time	0,006s
Start time	Fri Apr 03 18:55:18 CST 2026
Finish time	Fri Apr 03 18:55:18 CST 2026
Query	-- Iniciamos el proceso de exportación de datos
COPY (	
SELECT	
e.employee_number,	-- Número identificador del empleado
dc.department_name,	-- Nombre del departamento (desde el catálogo)
j.job_role_name,	-- Nombre del puesto (desde el catálogo)
e.monthly_income,	-- Ingreso mensual actual
e.job_satisfaction,	-- Nivel de satisfacción reportado
FROM employee_final AS e	
-- Unimos la tabla de empleados con los nombres de departamentos y puestos	
JOIN	
department_catalog AS dc	
ON e.department_id = dc.department_id	

Python (Jupyter) – Limpieza de duplicados y transformación

Cargar dataset exportado

```
#
=====
# Título: Procesamiento Inicial de Datos de Riesgo RRHH
#
# Objetivo: Cargar y validar la estructura del archivo CSV de
empleados.
#
# Descripción: Este script localiza un archivo específico de
recursos humanos
# (riesgo de fuga), lo lee usando pandas y muestra un resumen
técnico (info) y
# las primeras filas para asegurar que los datos se han cargado
correctamente.
#
# Resultado PNG: day02_empl_risk_data_cleaning.png
#
=====
=====

import pandas as pd # Biblioteca para manipular y analizar los
datos
import os           # Biblioteca para gestionar rutas de archivos en el
sistema operativo
from pathlib import Path # Biblioteca moderna para gestionar
rutas de archivos de forma independiente al SO

# -----
# 1. Configuración de rutas.
```

```

# -----

# Localizamos la carpeta raíz del proyecto subiendo 3 niveles
desde este archivo.
base_dir = Path(__file__).resolve().parents[3]

# Construye la ruta hacia la carpeta específica de datos
procesados de RRHH.
target_raw_dir = base_dir / '02_data' / 'processed' / 'rrhh'

# Crea la ruta final apuntando directamente al archivo CSV que
necesitamos.
rrhh_path_csv = target_raw_dir / 'employees_attrition.csv'

# -----
# 2. Carga y visualización de datos.
# -----

# Comprueba si el archivo físico existe en la ruta indicada para
evitar errores de lectura.
if rrhh_path_csv.exists():
    # Carga el contenido del CSV en una tabla de datos (DataFrame).
    df = pd.read_csv(rrhh_path_csv)
    # Informa al usuario que la carga fue exitosa y muestra cuántas
    filas tiene originalmente.
    print(f"✅ Archivo cargado con éxito. Filas iniciales: {len(df)}")
else:
    # Si el archivo no está, detiene el programa y lanza un mensaje
    de error claro.
    raise FileNotFoundError(f"❌ No se encontró el archivo en:
{rrhh_path_csv}")

# Imprime un resumen técnico de la tabla (nombres de columnas y
tipos de datos).
print("\nResumen técnico (datos crudos):")
print(df.info())

# Muestra las primeras 5 filas para una revisión visual rápida de la
información.
print("\nVista previa (datos crudos):")
print(df.head())

```



```

# -----
# 3. Eliminación de duplicados
# -----

# Identifica y borra filas repetidas basándose solo en el número de
# empleado.
# .copy() asegura que trabajamos sobre una tabla nueva e
# independiente.
df_clean = df.drop_duplicates(subset="employee_number").copy()

# Imprime el total de registros restantes para confirmar cuántos
# duplicados se borraron.
print(f"Registros tras eliminar duplicados: {df_clean.shape[0]}")

# -----
# 4. Transformación de tipos de datos
# -----

# Definimos un diccionario que asocia la columna con el tipo de
# dato deseado (entero).
cols_to_fix = {
    "monthly_income": int,
    "job_satisfaction": int
}

try:
    # Intenta aplicar el cambio de formato a las columnas del
    # diccionario de una sola vez.
    df_clean = df_clean.astype(cols_to_fix)
    print("\nTransformación de tipos completada con éxito.")
except KeyError as e:
    # Si alguna columna del diccionario no existe en la tabla, nos
    # avisa cuál falta.
    print(f"Error: No se encontró la columna {e}")

# -----
# 5. Validación final del proceso
# -----

# Muestra el resumen técnico final para confirmar que los cambios
# de tipo se aplicaron.
print("\nResumen técnico final:")
print(df_clean.info())

```

```
# Muestra una vista previa de la tabla ya limpia y transformada.
print("\nVista previa de datos limpios:")
print(df_clean.head())
```

▪ **Resultado PNG: day02_empl_risk_data_cleaning.png**

```
Directorio Base: C:\DM_Lab\DM_Roadmap_P1_120D\02_data\processed\rrhh\employees_attrition.csv
✅ Archivo cargado con éxito. Filas iniciales: 143
<class 'pandas.DataFrame'>
RangeIndex: 143 entries, 0 to 142
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   employee_number        143 non-null    int64
1   department_name        143 non-null    str
2   job_role_name          143 non-null    str
3   monthly_income         143 non-null    int64
4   job_satisfaction       143 non-null    int64
dtypes: int64(3), str(2)
memory usage: 5.7 KB
None
   employee_number  department_name  job_role_name  monthly_income  job_satisfaction
0              20  Research & Development  Manufacturing Director           9980              1
1              38              Sales              Manager          18947              1
2              52              Sales  Sales Executive           5376              1
3              68              Sales  Sales Executive           5454              1
4              70  Research & Development  Healthcare Representative           9884              1
Registros tras eliminar duplicados: 143

Transformación de tipos de datos completada con éxito.

Resumen Técnico final:
<class 'pandas.DataFrame'>
RangeIndex: 143 entries, 0 to 142
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   employee_number        143 non-null    int64
1   department_name        143 non-null    str
2   job_role_name          143 non-null    str
3   monthly_income         143 non-null    int64
4   job_satisfaction       143 non-null    int64
dtypes: int64(3), str(2)
memory usage: 5.7 KB
None

Vista previa de datos limpios:
   employee_number  department_name  job_role_name  monthly_income  job_satisfaction
0              20  Research & Development  Manufacturing Director           9980              1
1              38              Sales              Manager          18947              1
2              52              Sales  Sales Executive           5376              1
3              68              Sales  Sales Executive           5454              1
4              70  Research & Development  Healthcare Representative           9884              1
PS C:\Users\User\AppData\Local\Programs\Microsoft VS Code> █
```



Power BI – Cargar dataset / Visualización de KPIs



Paso 1: Cargar dataset exportado

- **Abrir Power BI Desktop:**
 - **Acción:**
 - Localiza el ícono de **Power BI Desktop** en tu menú de inicio o barra de búsqueda de Windows.
 - Haz clic para abrir la aplicación.
 - **Validación:**
 - La pantalla inicial debe mostrar el panel de bienvenida con la opción **Obtener datos (Get Data)**.
- **Seleccionar fuente de datos:**
 - **Acción:**
 - En la pantalla principal, haz clic en **Obtener datos (Get Data)**.
 - Se abrirá una ventana con múltiples opciones de fuentes de datos.
 - Selecciona **Texto/CSV** si exportaste el dataset como **CSV** (employees_attrition.csv).
 - Alternativamente, selecciona **PostgreSQL Database** si deseas conectarte directamente a la base **ibm_hr_analytics**.
 - **Validación:**
 - La ventana debe mostrar un cuadro de diálogo para elegir archivo **CSV** o configurar conexión SQL.
- **Importar archivo CSV:**
 - **Acción (CSV):**
 - Haz clic en **Examinar (Browse)**.
 - Navega hasta la carpeta donde guardaste el archivo **employees_attrition.csv**.
 - Selecciónalo y haz clic en **Abrir**.
 - **Validación:**
 - **Power BI** mostrará una vista previa del contenido del archivo con columnas y filas.
- **Configurar delimitadores y encabezados:**
 - **Acción:**
 - Verifica que el delimitador sea **coma (,)**.
 - Confirma que la opción **Encabezados de columna** esté activada (para que la primera fila se use como nombres de columnas).
 - **Validación:**
 - Las columnas deben aparecer con nombres correctos (**employee_number**, **monthly_income**, **job_satisfaction**, etc.).

- **Cargar datos al modelo:**
 - **Acción:**
 - Haz clic en **Cargar (Load)**.
 - **Power BI** importará el dataset y lo mostrará en el panel derecho bajo **Campos (Fields)**.
 - **Validación:**
 - Debes ver la tabla **employees_attrition** listada con todas sus columnas disponibles.
- **Conexión alternativa vía PostgreSQL:**
*(Si prefieres trabajar directo con la base de datos en lugar del **CSV**)*
- **Acción:**
 - Selecciona **PostgreSQL Database** en la ventana de fuentes.
- **Configura:**
 - **Servidor:** localhost
 - **Base de datos:** ibm_hr_analytics
 - **Usuario:** postgres
 - **Contraseña:** la definida en tu instalación.
- Haz clic en **Conectar (Connect)**.
- **Validación:**
 - **Power BI** mostrará las tablas disponibles (**employee_master_data**, **employees_final**, etc.).
 - Selecciona **employees_final** y haz clic en **Cargar (Load)**.



Paso 2: Crear KPI Attrition Rate vs Monthly Income

- **Seleccionar visualización:**
 - **Acción:**
 - En el panel de **Visualizaciones**, haz clic en el ícono de **gráfico de dispersión (Scatter chart)**.
 - Se añadirá un gráfico vacío al lienzo.
- **Configurar ejes:**
 - **Acción:**
 - Arrastra la columna **monthly_income** al eje **X**.
 - Arrastra la columna **job_satisfaction** al eje **Y**.
 - Arrastra la columna **attrition** al campo **Leyenda (Legend)** para diferenciar empleados que dejaron la empresa (Yes) de los que permanecen (No).
- **Validación inicial:**
 - **Acción:**
 - Debes ver puntos distribuidos en el gráfico, con colores distintos según el valor de **attrition**.

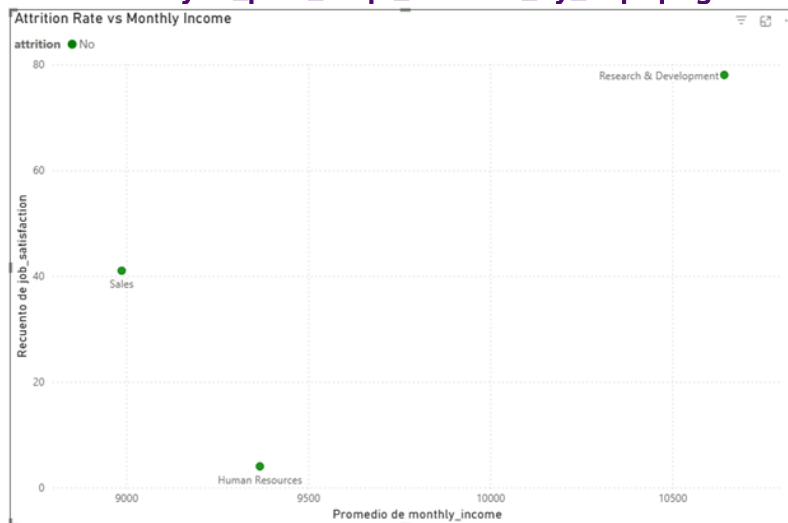
- El eje X debe mostrar ingresos, y el eje Y niveles de satisfacción.



Paso 3: Ajustar formato y presentación

- **Título del gráfico:**
 - **Acción:**
 - Haz clic en el gráfico.
 - En el panel de **Formato (Format)**, activa la opción **Título**.
 - Escribe: "Attrition Rate vs Monthly Income".
- **Etiquetas de datos:**
 - **Acción:**
 - En el panel de **Formato**, activa **Etiquetas de datos (Data labels)**.
 - Configura para mostrar department_name si deseas identificar departamentos específicos.
- **Paleta de colores corporativa:**
 - **Acción:**
 - En el panel de **Formato**, ajusta los colores de la leyenda:
 - Attrition = Yes → rojo.
 - Attrition = No → verde.
 - Esto facilita la interpretación visual.
- **Validación final:**
 - **Acción:**
 - El gráfico debe mostrar claramente la relación entre ingresos y satisfacción, diferenciando empleados que abandonaron la empresa.
 - KPI listo para análisis y discusión de insights.

Resultado PNG: day02_pbix_empl_attrition_by_dept.png



- **Insights:**
 - **Alerta de Retención: Talento sénior** altamente pagado en riesgo inminente de fuga por desconexión emocional.
 - **Ineficiencia:** El alto salario no garantiza la **retención**; la **cultura organizacional** y el **balance vida-trabajo** requieren atención urgente.
 - **Impacto: Empleados desmotivados** de alto nivel actúan como **detractores** internos, reduciendo la **productividad**.
 - **Acción:** RRHH debe priorizar **Stay Interviews** y **reconocimiento no monetario** para revertir la situación.



Sesión de Mentoría 1:1 (Resolución)

- **Pregunta guía:** ¿Por qué es más eficiente **normalizar** antes de construir dashboards?
- **Respuesta esperada:** Porque reduce **redundancia**, asegura **consistencia** y facilita **mantenimiento**.



Estrategia de Solución

- **Camino más corto y elegante:**
 - Crear **catálogos** primero.
 - Insertar **valores únicos** con **SELECT DISTINCT**.
 - Usar **FKs** en **tabla final**.
 - Validar **integridad**



Business Insights

- **Ejemplo:**
 - **Query:** empleados con **salario > 5000** y **baja satisfacción**.
 - **Insight:** "El grupo con **alto salario** pero **baja satisfacción** es crítico: **riesgo de rotación** en **talento clave**."
 - **Valor:** orientar **políticas de retención**.



Debrief de Errores (Top 3 Pitfalls)

- **Olvidar normalizar columnas** → genera **duplicados masivos**.
- **Prevención:** aplicar **SELECT DISTINCT** al poblar **catálogos**.
- **Usar nombres inconsistentes** → confusión en **queries**.
- **Prevención:** aplicar **snake_case** desde el inicio.
- **No validar integridad referencial** → **FKs** apuntando a **valores inexistentes**.
- **Prevención:** auditoría con **LEFT JOIN ... WHERE IS NULL**.